

cs5800-hw_3_exercises

Mingxi Zhu

May 2026

1 Exercise 2.3

Section 2.2 describes a method for solving recurrence relations which is based on analyzing the recursion tree and deriving a formula for the work done at each level. Another (closely related) method is to expand out the recurrence a few times, until a pattern emerges. For instance, let's start with the familiar $T(n) = 2T(n/2) + O(n)$. Think of $O(n)$ as being $\leq cn$ for some constant c , so: $T(n) \leq 2T(n/2) + cn$. By repeatedly applying this rule, we can bound $T(n)$ in terms of $T(n/2)$, then $T(n/4)$, then $T(n/8)$, and so on, at each step getting closer to the value of $T(\cdot)$ we do know, namely $T(1) = O(1)$.

$$\begin{aligned} T(n) &\leq 2T(n/2) + cn \\ &\leq 2[2T(n/4) + cn/2] + cn = 4T(n/4) + 2cn \\ &\leq 4[2T(n/8) + cn/4] + 2cn = 8T(n/8) + 3cn \\ &\leq 8[2T(n/16) + cn/8] + 3cn = 16T(n/16) + 4cn \\ &\vdots \end{aligned}$$

A pattern is emerging... the general term is

$$T(n) \leq 2^k T(n/2^k) + kcn.$$

Plugging in $k = \log_2 n$, we get $T(n) \leq nT(1) + cn \log_2 n = O(n \log n)$.

- (a) Do the same thing for the recurrence $T(n) = 3T(n/2) + O(n)$. What is the general k th term in this case? And what value of k should be plugged in to get the answer?
- (b) Now try the recurrence $T(n) = T(n-1) + O(1)$, a case which is not covered by the master theorem. Can you solve this too?

(a) Answer:

$$\begin{aligned} T(n) &\leq 3T(n/2) + cn \\ &\leq 3(3T(n/4) + cn/2) + cn = 9T(n/4) + 5cn/2 \\ &\leq 9(3T(n/8) + cn/4) + 5cn/2 = 27T(n/8) + 19cn/4 \\ &\leq \dots \\ &\leq 3^k T(n/2^k) + ((\frac{3}{2})^0 + (\frac{3}{2})^1 + \dots + (\frac{3}{2})^{k-1})cn \\ &\leq 3^k T(n/2^k) + (\sum_{i=0}^{k-1} \frac{3^i}{2})cn \end{aligned}$$

$k = \log_2 n$, because each level n is split ted by 2

(b) Answer:

Let $O(1) \leq c$, then

$$T(n) \leq T(n-1) + c$$

$$\begin{aligned}
&\leq (T(n-2) + c) + c = T(n-2) + 2c \\
&\leq (T(n-3) + c) + 2c = T(n-3) + 3c \\
&\leq \dots \\
&\leq T(1) + cn
\end{aligned}$$

We know $T(1) = O(1)$, $O(n) \leq cn$, therefore, $T(n) = O(n)$

2 Exercise 2.4

Suppose you are choosing between the following three algorithms:

- Algorithm A solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.
- Algorithm B solves problems of size n by recursively solving two subproblems of size $n-1$ and then combining the solutions in constant time.
- Algorithm C solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^2)$ time.

What are the running times of each of these algorithms (in big-O notation), and which would you choose? [note: (b) doesn't fit the Master Theorem; reason about it as a binary tree]

Answer:

- Algorithm A has *subproblem* = 5, *divisor* = 2, and linear combination, can be represented as $T(n) = 5T(n/2) + O(n^1)$.
According to Master Theorem, $a = 5, b = 2, d = 1$
then, $d \leq \log_b a = \log_2 5$
 $T(n) = O(n^{\log_2 5})$
- Algorithm B can be represented as $T(n) = 2T(n-1) + O(1)$
 $= 4T(n-2) + 3O(1)$
 $= \dots$
 $= 2^k T(1) + (\sum_{i=0}^{n-1} 2^i) O(1)$
We know it is a *depth* = n binary tree. The bottom level has 2^n times constant time operation.
 $T(n) = O(2^n)$
- Algorithm C can be represented as $T(n) = 9T(n/3) + O(n^2)$.
According to Master Theorem, $a = 9, b = 3, d = 2$
then, $d = \log_b a = \log_3 9$
 $T(n) = O(n^2 \log n)$

Choose Algorithm C, because $O(n^2 \log n) \leq O(n^{\log_2 5}) \leq O(2^n)$

3 Exercise 2.5(a)-(h)

[note: (g) and (h) don't have the form of the Master Theorem]

(a) $T(n) = 2T(n/3) + 1$

Answer: $a = 2, b = 3, d = 0, 0 < \log_3 2$. therefore, $T(n) = O(n^{\log_3 2})$

(b) $T(n) = 5T(n/4) + n$

Answer: $a = 5, b = 4, d = 1, 1 < \log_4 5$. therefore, $T(n) = O(n^{\log_4 5})$

(c) $T(n) = 7T(n/7) + n$

Answer: $a = 7, b = 7, d = 1, 1 = \log_7 7$. therefore, $T(n) = O(n \log n)$

(d) $T(n) = 9T(n/3) + n^2$

Answer: $a = 9, b = 3, d = 2, 2 = \log_3 9$. therefore, $T(n) = O(n^2 \log n)$

(e) $T(n) = 8T(n/2) + n^3$

Answer: $a = 8, b = 2, d = 3, 3 = \log_2 8$. therefore, $T(n) = O(n^3 \log n)$

(f) $T(n) = 49T(n/25) + n^{3/2} \log n$

Answer: $a = 49, b = 25, d = 3/2, 3/2 > \log_{25} 49$. therefore, $T(n) = O(n^{3/2} \log n)$

(g) $T(n) = T(n-1) + 2$

Answer: $T(n-1) + 2 = T(n-2) + 4 = \dots = T(1) + 2n$. therefore, $T(n) = O(n)$

(h) $T(n) = T(n-1) + n^c$, where $c \geq 1$ is a constant

Answer: $T(n-1) + n^c = T(n-2) + (n-1)^c + n^c = \dots = T(1) + \sum_{i=1}^n i^c$. According to Faulhaber's formula $\sum_{i=1}^n i^c$ is a $c+1$ polynomial, Therefore, $T(n) = O(n^{c+1})$

4 Acknowledgements

4.0.1 Resources

Master Theorem, Page 60, Dasgupta et. al. text. Faulhaber's formula, Wikipedia.

4.0.2 LLM Disclosure

Study how to use different functions in Overleaf

5 Reflection

Reinforced my understanding of Master Theorem. Strengthen the concept of recursion algorithms and its time complexity analysis.